

Lifecycle-Governed Retrieval for Action-Safe Enterprise Knowledge Graphs

Amith Reddy Ravuru
Independent Researcher

San Francisco Bay Area, California, USA

Abstract—In enterprise knowledge graph retrieval systems, relevant evidence is not necessarily valid for the action it is used to justify. A retrieved fact may be outdated, replaced by a newer record, or incompatible with the policy governing the decision. This paper formalizes lifecycle-governed retrieval (LGR), a decision procedure that connects evidence retrieval to action authorization. It represents facts as assertion events carrying validity periods, versions, and source information. LGR first checks whether evidence is eligible for the requested decision, then resolves conflicting claims using declared source authority and record time, and finally checks whether policy and workflow rules permit the action. Only registered actions can be admitted; missing required authorization or evidence, or an unresolved conflict in evidence required for the action, prevents admission. Under stated assumptions, we establish that admitted actions that change external state satisfy the required policy and workflow conditions and rely on eligible evidence concerning the intended action target. The procedure records supporting evidence and reasons for its decisions, allowing the same decision record to be reproduced from an unchanged graph and governance snapshot. We provide an executable reference implementation and EvoMem-Enterprise, a synthetic benchmark covering changing facts, policies, and workflows. Pilot evaluations and targeted tests examine both retrieval quality and compliance with the specified decision rules, exposing failures that aggregate retrieval scores alone can miss. The contribution is an explicit, testable integration of evidence eligibility, conflict resolution, and action authorization, rather than a new retrieval algorithm. The evaluation supports this bounded systems design; it does not establish production safety or effectiveness with a deployed language model.

Index Terms—knowledge graph systems, enterprise memory, temporal knowledge graphs, provenance, ontology evolution, policy governance, workflow constraints, retrieval-augmented generation

I. INTRODUCTION

Retrieval-augmented generation (RAG) grounds language models in external evidence [1], and graph-centric retrieval extends that evidence to entities, relations, neighborhoods, and multi-hop structures [2], [3], [4], [5]. This suits enterprise assistance, where a response may depend on a customer, a contract, an entitlement, a policy, and a running workflow. Yet relevance ranking leaves a systems question unanswered: *may the retrieved evidence support the requested decision now?*

The distinction matters when an assistant proposes an action rather than summarizes. Consider three cases: a support assistant retrieves a superseded entitlement; a service agent applies a service-level agreement (SLA) tier from the wrong policy version; a refund assistant finds a policy-eligible refund whose workflow has not reached the state permitting it. A

related real-world failure illustrates the consequences of policy-inconsistent assistance: in *Moffatt v. Air Canada* the B.C. Civil Resolution Tribunal held an airline liable after its chatbot answered a refund question with conditions contradicting the applicable bereavement-fare policy [6]. Fluent grounding is insufficient: evidence must also be current, attributable, semantically compatible, conflict-resolved, and action-admissible.

Prior work provides graph-based retrieval and associative memory [2], [3], [7], temporal and event-centered representations [8], [9], [10], [11], [12], conflict and evidence-governance mechanisms [13], [14], [15], and process-aware retrieval [16], [17]. These do not by themselves define one deterministic contract ordering lifecycle, provenance, conflict, policy, and workflow checks before evidence can authorize an action. The composition carries design content no component forces: values must be normalized before conflicts can be grouped, guards must stay outside eligibility so evidence cannot authorize itself, and decision evidence must be completed beyond the top- k context before any guard runs.

LGR treats each fact as a versioned, time-bounded assertion event rather than a static triple: each separates a validity interval from a record time and carries lifecycle state, ontology and policy versions, provenance, and supersession links. LGR filters assertions before action evaluation, resolves conflicts after normalization, and applies a registered default-deny gate; blocked items carry reasons, admitted effectful actions carry table-derived witnesses.

The paper makes four bounded contributions:

- 1) A knowledge graph (KG) systems architecture composing lifecycle eligibility, conflict resolution, governance tables, and action admission into a replayable decision packet, realized as an executable operator.
- 2) A formal, closed-world action model with a default-deny registry, a proposition that any admitted effectful action is registered, target-bound, guarded, evidenced, and conflict-free under stated assumptions, and a determinism proposition for its decision record.
- 3) EvoMem-Enterprise, a generated enterprise benchmark spanning ontology evolution, policy versions, provenance, supersession, contradictions, and workflow state.
- 4) A three-layer evaluation methodology separating retrieval correctness, decision conformance, and model behavior, instantiated by a validation-only label checker, a conformance suite with adversarial counterexamples, decisive strata, and a frozen pilot.

Scope of claims. Every empirical result here is a controlled design-and-pilot evaluation: the frozen pilot uses one generated world, and the pre-specified extension uses ten test worlds from the same generator family; both assume the supplied tables are correct, use a deterministic proxy in place of a language model, and include targeted branch testing. The paper claims a falsifiable specification, an executable reference behavior, and benchmark-level feasibility evidence—neither production readiness nor superiority over the systems in Section II; Section VII structures the residual threats.

II. RELATED WORK AND NOVELTY BOUNDARY

GraphRAG, KG²RAG, GraphFlow, and G-Retriever adapt the RAG pattern [1] to graph topology, knowledge subgraphs, and graph-structured reasoning [2], [3], [4], [5], addressing candidate selection or answer synthesis. LGR treats candidate generation as pluggable and imposes admission constraints afterward: relevance cannot compensate for an expired assertion, an incompatible mapping, or an unregistered action. HippoRAG is complementary associative-memory work, addressing how a generator retrieves long-term memory rather than whether that memory may support an action at decision time [7].

The temporal core is deliberately standard and LGR renames none of it: separating record time from validity interval is the transaction-time/valid-time distinction of bitemporal databases [18], standardized in SQL:2011 [19] and given semantic-web form by temporal RDF [20]; TimeOK is a textbook as-of predicate. Temporal KG research models time-varying events [12], and recent work brings entity-event graphs, time-sensitive retrieval, and document lineage to RAG and agent memory [9], [10], [11], [8], [21]. Against this lineage LGR’s delta is narrow: governance versions—ontology, policy, trust-table—participate in admission alongside bitemporal visibility, which none of these lines defines. Likewise its local closed-world decision rule adapts the local closed-world assumption of KB completion [22]; the asymmetric variant, in which unknown world facts stay unknown while unwitnessed authorizations become false, is this paper’s framing.

Conflict-aware RAG studies disagreements between retrieved and parametric information or among changing sources [13], [14]; LedgerRAG keeps a claim-level ledger, adjudicates by authority and time, and can hold unresolved claims for review [15]; ordering conflicting defaults by priority while leaving incomparable residues undecided is prioritized defeasible reasoning [23]. LGR claims no novelty for ledgers, authority-first adjudication, or the resolution logic: its resolver is a table-driven instance of prioritized defaults whose distinguishing behavior is that tied maxima stay unresolved and the gate treats that residue as blocking, per decision variable.

Process-aware retrieval has been explored for process mining and legal-information workflows [16], [17], and runtime guardrails check agent actions against safety policies [24]. LGR’s workflow guard treats an externally modeled process such as BPMN [25] as a governance table consulted at admission time, not a retrieval target, and its gate differs from action-policy guardrails in what it consults: admission depends

on the eligibility and conflict status of the supporting evidence, so a policy-permitted action is still blocked when that evidence is stale, unresolved, or unwitnessed. Remaining fields build on PROV-O [26], RDF-star [27], SHACL [28], and XACML-style vocabularies [29]; one baseline follows event sourcing [30].

Overall, LGR proposes no new retrieval, memory, temporal, conflict, or process method: its contribution is one compositional decision interface that knows which assertions were excluded and why, whether a conflict was resolved, which bindings authorize an action, and whether the decision reproduces from the same tables and graph state. It claims no learned retriever, no universal method for resolving truth, no organization-wide policy correctness, and no safe autonomous execution.

III. SYSTEM AND FORMAL MODEL

A. Assertion-event knowledge graph

Let $G = (V, E, \text{type}, \text{attr})$ be a typed graph with assertion-event nodes $A \subseteq V$; version, policy, workflow, source, and provenance nodes also lie in V . Each assertion event extends a subject–predicate–object statement with timestamps, versions, provenance, and lifecycle status:

$$a = \langle \tau, \sigma, \theta, \nu, \pi, \lambda, \psi \rangle, \quad (1)$$

where $\tau = (s, p, o)$ is the subject, predicate, and object; $\sigma = (\sigma_e, \sigma_r)$ the entity and relation scope; $\theta = (t_r, [v_f, v_u])$ the record time (transaction time: when the system learned it [18]) and validity interval (valid time: when the fact held); $\nu = (\omega, \rho)$ the ontology and policy versions; $\pi = (\gamma, \mu)$ the provenance record and originating source; $\lambda \in \{\text{active}, \text{superseded}, \text{retracted}\}$ the lifecycle state; and ψ an optional workflow binding. An implementation may also store the world-event time as metadata, as the released benchmark does; the predicates consume world time only through the validity interval, so the tuple omits it.

Supersession is a directed relation $R_{\text{sup}} \subseteq A \times A$ rather than a tuple field, with $\text{SupersededBy}(a) = \{a' \in A : (a, a') \in R_{\text{sup}}\}$; an edge (a, a') means a' explicitly replaces a . A replacement takes effect once its record time is no later than t_q , and its later expiry or retraction does not revive a : a new value requires a new event. Dangling, cross-subject, and cyclic links are rejected.

The model separates assertion status, historical visibility, and removal policy. *Supersession* is induced by a visible replacement and carries its record time through R_{sup} , so visibility follows from the log with no separate flag: a claim replaced on 1 June is visible as of 1 May and hidden as of 1 July. *Retraction* is a removal decision, such as a data-erasure obligation, and applies in every mode, so a record retracted on 1 July is unavailable even as of 1 May. That asymmetry is why LifeOK is mode-scoped and why retraction carries no timestamp. Storage is unconstrained: metadata may attach to graph nodes, annotate RDF statements via RDF-star, or live in an event store.

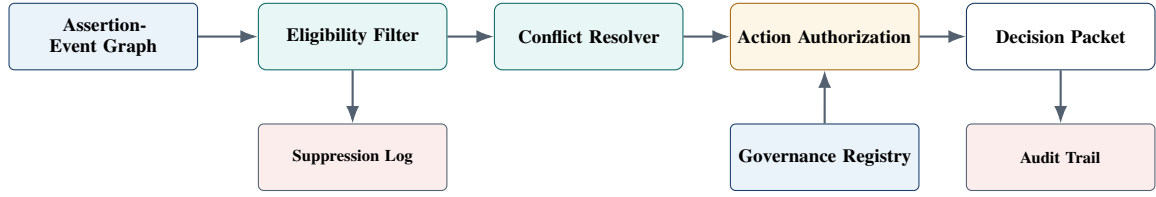


Fig. 1. LGR architecture. Evidence flows left-to-right; the registry feeds authorization; filter and packet emit auditable outputs.

A decision query is

$$q = \langle q_e, q_r, t_q, m_q, \beta_q, \omega_q, \rho_q, w_q, X_q, \mathcal{K}_q \rangle, \quad (2)$$

which separates entity/relation scope q_e, q_r , decision time t_q , query mode $m_q \in \{\text{current}, \text{historical}\}$, an optionally declared action target β_q , target versions ω_q, ρ_q , workflow instance/state w_q , candidate actions X_q , and the action-critical normalized keys $\mathcal{K}_q = \bigcup_{x \in X_q} \text{Crit}(x)$, which are derived from the registry rather than supplied independently. Throughout, T denotes the versioned governance bundle: the action registry, policy-family bindings and rules, workflow models and transitions, source catalog and relation-specific trust order, ontology mappings, provenance-activity registry, relation-semantics table, and snapshot metadata.

The underlying KG need not adopt global closed-world semantics. LGR applies a *local closed-world decision rule* to the snapshot available for q : absence of an assertion leaves a world fact unknown, but absence of a required registration, binding, or witness makes an effectful authorization false, so incomplete evidence yields abstention rather than permission by open-world inference.

B. Staged assertion eligibility and conflict resolution

Trust is a versioned governance input. For policy version ρ_q , let

$$\text{Trust}_{\rho_q} : (\hat{p}, \mu) \rightarrow \mathbb{Z} \quad (3)$$

map a normalized relation and source to an authority tier. A governance owner builds it from a versioned source catalog and relation-specific precedence rules; the validator rejects duplicate assignments, missing tiers for declared action-relevant pairs, and a version inconsistent with ρ_q . Relations may use different orders, whose correctness is an organizational assumption.

Normalization maps a predicate to at most one decision variable. The mapping table declares identity, rewrite, and split entries per ontology-version pair, and distinguishes renaming classes from splits: several names may denote one variable across versions (`support_plan`, `entitlement_profile`, `support_entitlement` here), each class having one canonical name—the name the trust and relation-semantics tables declare. $\text{Norm}_{\omega_q}(p)$ collects the relations p can reach, canonicalizes them, and returns the result if exactly one remains. If a split reaches two canonical names, the mapping is *ambiguous* and the assertion is ineligible (`ontology-ambiguous`): resolving it by the order the query listed its relations would let syntax decide what a value means, so a value-transforming split must be

declared as a rewrite per target relation. Canonicalization also stops a conflict hiding behind a rename, since two records under different names normalize to one key and are compared.

$\kappa_q(a)$ denotes a normalized *decision-variable* key such as (*customer*, *subscription_status*); the asserted object is deliberately not part of it. A versioned relation-semantics table declares whether distinct values are incompatible: single-value relations resolve, multi-value relations coexist. Because κ_q is keyed by the canonical relation and the validator requires a trust order per canonical relation, tiers are always compared within one order, never across scales. Write $\kappa_q(Y) = \{\kappa_q(a) : a \in Y\}$.

For a query q , a candidate assertion must satisfy eight predicates before it can be used by an action guard:

$$\begin{aligned} \text{Eligible}(a, q) \Leftrightarrow & \text{ScopeOK} \wedge \text{TimeOK} \wedge \text{LifeOK} \\ & \wedge \text{OntOK} \wedge \text{PolicyVerOK} \wedge \text{ProvOK} \\ & \wedge \text{SourceOK} \wedge \text{SupOK}. \end{aligned} \quad (4)$$

Eligibility is checked against the *decision-evidence scope* $\hat{q}_r$ — q_r plus every relation the candidate actions' guards declare—not the retrieval scope q_r , so that narrowing a retrieval scope cannot silently change an authorization. Writing $t_r(a')$ for the record-time component of a' , and taking Match_{ω_q} over $\hat{q}_r$, the predicates abbreviate the following checks against the versioned table bundle:

$$\begin{aligned} \text{ScopeOK}(a, q) & \Leftrightarrow \text{Match}(\sigma_e, q_e) \wedge \text{Match}_{\omega_q}(\sigma_r, \hat{q}_r), \\ \text{TimeOK}(a, q) & \Leftrightarrow t_r \leq t_q \wedge t_q \in [v_f, v_u), \\ \text{LifeOK}(a, q) & \Leftrightarrow \lambda = \text{active}, \\ \text{OntOK}(a, q) & \Leftrightarrow \omega = \omega_q \vee \text{Map}_{\omega \rightarrow \omega_q}(p) \neq \perp, \\ \text{PolicyVerOK}(a, q) & \Leftrightarrow \neg \text{GovFact}(a) \vee \rho = \rho_q, \\ \text{ProvOK}(a, q) & \Leftrightarrow \gamma \neq \perp \wedge \gamma \in \text{dom}(\Gamma_T) \wedge \Gamma_T(\gamma) = \mu, \\ \text{SourceOK}(a, q) & \Leftrightarrow \text{Trust}_{\rho_q}(\hat{p}, \mu) \neq \perp, \\ \text{SupOK}(a, q) & \Leftrightarrow \nexists a' \in \text{SupersededBy}(a) : t_r(a') \leq t_q. \end{aligned} \quad (5)$$

$\text{Match}(\sigma_e, q_e)$ holds when the assertion's subject or entity scope meets the query's, or its workflow instance is the query's; $\text{Match}_{\omega_q}(\sigma_r, \hat{q}_r)$ when $\text{Norm}_{\omega_q}(p)$ is defined and lies in the canonical image of $\hat{q}_r$; $\perp$ is an undefined mapping. Γ_T is the declared provenance-activity registry, carried in the governance bundle and held apart from the assertion stream so a citation cannot vouch for itself; ProvOK fails on a citation absent from $\text{dom}(\Gamma_T)$ or registered to another source. Stating the condition as membership in Γ_T , rather than as a lookup attempted only when Γ_T is populated, makes an absent or empty registry withhold evidence rather than silently admit it. A timestamp present but unparseable fails eligibility rather than reading as an open boundary. A visible successor permanently suppresses its predecessor while an unrecorded future one does not, and

GovFact identifies policy and workflow assertions. Historical queries ($m_q = \text{historical}$) are evaluated at their audit time with LifeOK relaxed to $\lambda \neq \text{retracted}$, so superseded lineage stays explainable while a retracted record stays unavailable in every mode; by ModeOK they cannot authorize. The implementation records every failed predicate in a reason-coded suppression ledger. Crucially, neither a policy-action guard nor a workflow-action guard belongs to Eq. (4): this avoids circular eligibility in which an action selects the evidence needed to authorize itself.

Candidate retrieval is not the evidence boundary. $\text{Complete}(G, q, T)$ returns all snapshot assertions matching the query entity or workflow scope and any declared query, guard, conflict, or supersession dependency, records a completion manifest, and blocks rather than authorizes on a failed lookup. The first-stage set is

$$B_0(q) = \{a \in \text{Complete}(G, q, T) : \text{Eligible}(a, q)\}. \quad (6)$$

For each single-value relation LGR groups normalized assertions by $\kappa_q(a)$ and orders distinct values lexicographically by $(\text{Trust}_{\rho_q}(\hat{p}, \mu), t_r)$, descending, retaining all top assertions supporting one winning value. $\text{Losers}(q)$ is the eligible assertions supporting non-winning values of a uniquely resolved component, $\text{Unresolved}(q)$ those in components whose incompatible maximum is tied. Multi-value relations bypass the resolver. A tied maximum is *unresolved*: its members pass eligibility but are excluded from the decision-evidence set, so no member can witness an action:

$$B(q) = B_0(q) \setminus (\text{Losers}(q) \cup \text{Unresolved}(q)). \quad (7)$$

This is a reproducible protocol over an observed record, not a claim to determine truth.

Worked decision sequence, executable against the released tables. A query targeting ω_3 finds a `support_entitlement=Premium` record from `crm` and a later `Standard` record from `billing`; the case state under the ω_1 name `refund_status=approved`; a suspended subscription; and an ω_2 `entitlement_profile=Gold` record. `refund_status` reaches `workflow_state` by a rewrite chain and canonicalizes to that one variable, so the case state is eligible under its current name. The `entitlement_profile` record is not: its mapping splits into three distinct variables, and since normalization never consults the query, naming one cannot select a branch (ontology-ambiguous). The two `support_entitlement` records disagree on a single-value relation and `billing` outranks `crm`, so the `crm` record is a contradiction-loser. The transition `approved→refund_issued` is enabled, but the refund rule also requires a non-suspended subscription, so `issue_refund` is blocked guard-failed with the enabled transition recorded beside the failure.

C. Registered default-deny action admission

Registry and bindings. Let X be the action identifiers and $\text{Reg} : X \rightarrow \{\text{advisory}, \text{effectful}\}$ the registry; an unregistered

action is unavailable, not merely low-ranked. Bind_P and Bind_W map actions to versioned policy families and workflow models, and $\text{Bound}(x, q)$ holds when every binding the query invokes is present and family-compatible, so a same-named rule from another family cannot authorize x . Write $\text{Registered}(x) \Leftrightarrow x \in \text{dom}(\text{Reg})$.

Guards, witnesses, and the action target. A *guard atom* is one comparison in a policy rule or workflow transition, such as `workflow_state = approved`; its *witness* is the eligible assertion whose value satisfies it. Because eligible facts about different principals do not compose, an action is evaluated against one *action target* $\beta(q) = (\text{subject}, \text{workflow instance})$, and only assertions bound to it may witness x : those whose subject or entity scope contains the target or whose workflow instance is the target's, plus governance facts, which are not entity-scoped.

$\beta(q)$ is resolved from declared relationships only. It is the query's declared target β_q when one is given; otherwise, when q names a workflow instance w_q , the target's members are the entities that w_q 's own workflow assertions declare as participants, intersected with q_e . Only assertions carrying w_q from a source the trust table admits for `workflow_state` may declare them, so no record can enrol an entity into a case it does not belong to, and membership is never inferred from the shape of an identifier—spelling is a naming convention, not an authorization relation. When no declared relationship resolves the scope to one target, the action is refused (ambiguous-action-binding) rather than evaluated against a mixture. $\text{BindOK}(x, q)$ requires $\beta(q)$ to resolve to one target with every witness bound to it.

Conflict dependence and query mode. Let $\text{Crit}(x)$, declared in x 's registry entry, be the decision-variable keys whose unresolved status must block x ; with U the assertions in unresolved components, so $\kappa_q(U)$ is their key set, $\text{CritOK}(x, q) \Leftrightarrow \text{Crit}(x) \cap \kappa_q(U) = \emptyset$. The mode $m_q \in \{\text{current}, \text{historical}\}$ is an explicit model input with $\text{ModeOK}(x, q) \Leftrightarrow \text{Advisory}(x) \vee m_q = \text{current}$: a historical query explains an earlier decision and never authorizes a new effectful one. Let $P(x, q, B)$ mean a family-compatible policy permit with no equal-or-higher-priority satisfied deny, and $W(x, q, B)$ an enabled workflow transition. Admission is then

$$\text{Admit}(x, q, B) \Leftrightarrow \text{Registered}(x) \wedge \text{ModeOK}(x, q) \wedge \text{CritOK}(x, q) \wedge \text{GovernedOK}(x, q, B), \quad (8)$$

$$\text{GovernedOK}(x, q, B) \Leftrightarrow \text{Advisory}(x) \vee [\text{Effectful}(x) \wedge \text{Bound}(x, q) \wedge \text{BindOK}(x, q) \wedge P(x, q, B) \wedge W(x, q, B)], \quad (9)$$

An absent registration, binding, target, rule, transition, state, or witness makes an effectful action false. The output is an admitted action with its witnesses or a blocked one with a reason code, never a guessed authorization; each admitted action stores its witnesses with the rule and transition actually selected, so evaluation replays. The admitted set is $\text{Adm}(q, B) = \{x \in X_q : \text{Admit}(x, q, B)\}$.

The first proposition is a structural invariant of the definitions under its assumptions.

Proposition 1 (Structural effectful-action soundness). *Assume an immutable supplied snapshot whose declared dependencies*

are returned by Complete, complete registry and binding tables for that snapshot, correct table-driven guard evaluation, a validated deterministic relation-semantics table, a deterministic conflict resolver that leaves tied maxima unresolved, and witnesses drawn only from $B(q)$. Each effectful $x \in \text{Adm}(q, B(q))$ is registered, arises from a current-decision query, has a satisfied policy permit and workflow transition, has no equal-or-higher-priority satisfied deny, is supported by witnesses that satisfy Eq. (4) and are bound to one action target, and depends on no unresolved critical conflict.

Proof. Equation (8) excludes an unregistered action, a non-current query mode, and an unbound effectful action, and requires both satisfied guards. By Eqs. (6)–(7), guard witnesses are a subset of $B(q)$ and are therefore eligible, conflict winners, and not unresolved; BindOK additionally restricts them to assertions bound to the single target $\beta(q)$. The critical-key conjunct blocks any action depending on an unresolved component. $\square$

Proposition 1 is definitional: it verifies that the contract cannot emit an effectful action lacking a required condition when operator and tables behave as assumed—not that an implementation is defect-free or that an organization’s tables encode its policy. Its BindOK and ModeOK conjuncts are load-bearing: without the first, eligible facts about different principals compose into a justification no principal supports; without the second, an investigation query authorizes what it was meant to explain (Section VI).

Determinism is claimed for the *decision record*—completed evidence identities, suppression reasons, the conflict outcome, and each action’s verdict with its witnesses and selected rule and transition—not for a whole packet, which also carries the retrieval context R and runtime measurements such as latency.

Proposition 2 (Determinism of the decision record). *Fix a snapshot (G, T) , a query q , and a feature configuration. Completion, predicate evaluation, normalization, the lexicographic resolver, and the table-driven gate are each pure functions of their inputs, with ties that no rule orders emitted as unresolved rather than broken arbitrarily. Two evaluations therefore yield an identical decision record, and hence an identical record digest, whatever candidate generator supplied R .*

Proof. Each stage is a pure function of (G, T, q) , the configuration, and prior stages’ outputs, and none consumes R : completion is defined over the snapshot and declared dependencies, not over the retrieval context. A composition of pure functions is pure, so the record is uniquely determined. $\square$

D. Packet contract and evaluation layers

Algorithm 1 fixes the order, emitting $\Pi = (R, C, B, L, U, \text{Actions}, \text{Trace}, D, h)$, in which the completed set C , unresolved components U , suppression ledger L , digest D , and checkpoint h are explicit outputs, not internal state; Fig. 1 gives the architecture. A candidate generator—lexical, vector, graph-based, or hybrid—sits outside the safety boundary: it

Algorithm 1 Lifecycle-Governed Retrieval

Require: graph G , query q (with mode m_q), governance bundle T
Ensure: decision packet Π

```

1:  $R \leftarrow \text{CandidateAssertions}(G, q)$   $\triangleright$  context, not the boundary
2:  $C \leftarrow \text{Complete}(G, q, T)$ ;  $B_0 \leftarrow \emptyset$ ;  $L \leftarrow \emptyset$ 
3:  $\text{Actions} \leftarrow \emptyset$ ;  $\text{Trace} \leftarrow \emptyset$ 
4:  $D \leftarrow \text{SHA256}(\text{Canonicalize}(G, T))$ ;  $h \leftarrow \text{AssertionLogCheckpoint}(G)$ 
5: for all  $a \in C$  do
6:    $r \leftarrow \text{FailureReasons}(a, q, T)$   $\triangleright$  Eqs. (4)–(5)
7:   if  $r = \emptyset$  then  $B_0 \leftarrow B_0 \cup \{a\}$ 
8:   else  $L \leftarrow L \cup \{(a, r)\}$ 
9:   end if
10: end for
11:  $(B, U) \leftarrow \text{ResolveConflicts}(B_0, \kappa_q, \text{Trust}_{p_q})$   $\triangleright$  Eqs. (6)–(7)
12:  $\beta \leftarrow \text{ActionTarget}(q, B, T)$   $\triangleright$  membership declared by  $\text{Trust}$ -eligible records only
13: for all  $x \in X_q$  do
14:   if  $\neg \text{Registered}(x)$  or  $[Effectful(x) \wedge \neg \text{Bound}(x, q)]$  then  $\triangleright$  registry
15:      $\text{Actions}[x] \leftarrow \text{Block}(x, \text{unbound})$ 
16:   else if  $Effectful(x) \wedge m_q \neq \text{current}$  then  $\triangleright$  Eq. (8): mode
17:      $\text{Actions}[x] \leftarrow \text{Block}(x, \text{non-authorizing-query-mode})$ 
18:   else if  $Effectful(x) \wedge \neg \text{Resolves}(\beta)$  then  $\triangleright$  one action target
19:      $\text{Actions}[x] \leftarrow \text{Block}(x, \text{ambiguous-action-binding})$ 
20:   else if  $\text{Crit}(x) \cap \kappa_q(U) \neq \emptyset$  then  $\triangleright$  critical conflict
21:      $\text{Actions}[x] \leftarrow \text{Block}(x, \text{unresolved-conflict})$ 
22:   else if  $\neg \text{GovernedOK}(x, q, B)$  then  $\triangleright$  Eq. (9)
23:      $\text{Actions}[x] \leftarrow \text{Block}(x, \text{guard-failed})$ 
24:   else
25:      $\text{Actions}[x] \leftarrow \text{Admitted}(x, \text{Witnesses}_\beta(x, q, B, T))$   $\triangleright$  witnesses bound to  $\beta$ 
26:   end if
27:    $\text{Trace} \leftarrow \text{Trace} \cup \{(x, \text{Actions}[x], \text{Rule}(x), \text{Transition}(x))\}$ 
28: end for
29: return  $\Pi = (R, C, B, L, U, \text{Actions}, \text{Trace}, D, h)$   $\triangleright C, U$ , digest explicit

```

can improve the retrieval context but cannot emit an admissible action, because completion, eligibility, resolution, and the gate all run over the snapshot, not over its output. The packet serializes accepted assertions with versions and provenance, excluded ones with reason codes, conflict components with winners and unresolved flags, action candidates with bindings and guard witnesses, and a snapshot identifier: a SHA-256 digest over canonical ledger and table serializations plus the log checkpoint. A downstream large language model (LLM) may summarize or seek confirmation but must not treat absent evidence as approval.

This separates three often conflated layers: *retrieval correctness* (does the context a method returns carry the assertion events that $B(q)$ permits?), *decision conformance* (does the evaluator implement Eqs. (4)–(8)?), and *model behavior* (does an LLM use the packet correctly?). The original pilot and a pre-specified multiworld extension evaluate the first; targeted counterexamples and conformance tests evaluate selected branches of the second (Section VI); no language model is run for the third.

IV. EVO MEM-ENTERPRISE BENCHMARK

EvoMem-Enterprise stresses lifecycle and governance cases without exposing canonical timelines to retrieval. Following the reproducible-scenario spirit of temporal graph benchmarks [31], it models customer support, entitlement, SLA escalation, and refund decisions: each scenario has a canonical state timeline and produces an observed assertion-event stream with delayed

TABLE I
EVO-MEM-ENTERPRISE COMPOSITION. COUNTS DESCRIBE A GENERATED BENCHMARK, NOT AN OPERATIONAL DEPLOYMENT.

Artifact	Count
Entity-like nodes	601
Assertion events / state changes	611 / 215
Registered provenance activities	600
Ontology versions / policy artifacts	3 / 4
Workflow models / contradiction groups	2 / 60
All evaluation tasks / frozen test tasks	140 / 100
Action-labeled tasks (test / dev / smoke)	40 / 15 / 5
Frozen tasks by label regime (satisfiable / retro. / out-of-scope)	75 / 22 / 3

recordings, stale facts, supersessions, ontology migration, policy changes, source-trust variation, contradictions, and workflow-state cases. Gold labels record eligible evidence, exclusions, conflict outcomes, packet support, and action expectations. Each assertion also carries a contradiction-group identifier as gold instrumentation only; the operator derives contradiction semantics from the versioned relation table and never reads it.

Table I summarizes the release; development and smoke tasks support debugging, not tuning. All frozen tasks share one world’s entities, contradiction groups, and workflow instances, and action metrics use only the 40 frozen action tasks, so one task moves an action match rate by 2.5 points. No temporal-QA or versioned-document benchmark can anchor this externally, because none carries the governance surface the operator consumes—versioned trust orders, policy families, transition tables, registry bindings [31], [21]—and a hand-mapped slice would invent exactly the tables under test. Action labels are derived from the canonical state and checked by a separately coded table interpreter that does not import the evaluated LGR operator; all 60 agree with the versioned registry, policy rules, and workflow tables, which is a consistency check within one generated world, not independent validation of the tables.

The multiworld extension splits a master seed deterministically into per-stream generators: three development worlds support implementation checks, two validation worlds select the primary local comparator, and ten previously unevaluated test worlds provide the reported robustness result. Every generated world is retained, and each must pass structural, action-label, and evidence-label validation before evaluation. The version-3 labeling specification keeps retrospective and out-of-scope cases as diagnostic populations instead of deleting them; the primary population contains only current-decision tasks whose target is a minimal sufficient, contract-permitted evidence set. This changes the evaluation protocol, not the historical frozen release summarized in Table I. Seed-derivation mechanics, overwrite protections, and file organization are documented with the artifact.

A. Methods and metric definitions

All frozen comparisons use $k = 8$ retrieval-context items. The implemented generators are lexical top- k , recency-weighted retrieval, a 60-day time-to-live (TTL) filter, temporal-only KG retrieval, provenance-aware KG retrieval, ontology-versioned

retrieval, a GraphRAG-style entity-seeded expansion, and an event-sourced latest-state proxy [30]; LGR enables validity, life-cycle, provenance/trust, ontology/policy version, supersession, contradiction, and workflow features. These names identify released benchmark implementations, not reproductions of the systems in Section II.

Let $R(u)$ be the retrieval context a method returns for task u (at most k identifiers) and $E(u)$, $S(u)$, $L(u)$ the gold eligible, stale-or-superseded, and contradiction-loser sets. Evidence metrics are computed on $R(u)$, not on the completed evidence, and macro-averaged. Eligible precision and recall are $|R \cap E|/|R|$ and $|R \cap E|/|E|$; F1 is their per-task harmonic mean. *Stale non-exposure* is $|S \setminus R|/|S|$ and *loser non-exposure* is $|L \setminus R|/|L|$ (each 1 when its gold set is empty): the per-task fraction of gold-flagged items kept out of the context. Both are reported beside eligible recall, since a method returning nothing attains 1.000 on each and only the pair separates withholding stale evidence from withholding all of it. Tasks are partitioned by label regime: *satisfiable* when every assertion in $E(u)$ was recorded at or before the query instant, satisfies ScopeOK, and survives conflict resolution, so returning exactly $E(u)$ is contract-permitted; *retrospective* when some assertion in $E(u)$ was recorded later; *out-of-scope* when all were visible but some lies outside $\hat{q}_r$ or q_e . A validation-only interpreter checks eligibility and conflict survival without importing the evaluated operator or generator. Action-set agreement is the per-task indicator that the predicted valid-action set equals the gold set over the 40 frozen action tasks. The budget $k = 8$ never truncates a perfect selector, since no gold evidence set exceeds four assertions. LGR fills its context from its own completed, eligible, conflict-resolved set, ranked by requested-relation match, authority tier, and record time, appending witness identifiers the cutoff would drop; baselines fill theirs by their own scoring.

Frozen-pilot intervals are percentile intervals from 1,000 paired task-level bootstrap resamples; the multiworld primary interval instead uses 10,000 paired resamples of whole test worlds, preserving every method’s observations within a sampled world. For every main comparison the same completion and admission function follows the initial retrieval context, so a $k = 8$ cutoff cannot change an action decision: action-set agreement is a conformance check for the common gate, not a measure on which generators can claim an advantage.

V. EVALUATION RESULTS

A. Frozen pilot

The frozen split holds incompatible notions of a correct evidence set, so averaging them yields a target no behavior can maximize. Of the 100 tasks, 75 are *satisfiable*—the contract permits returning exactly the gold set—22 are *retrospective* (14 audit, 8 of 14 contradiction), requiring evidence recorded after the query instant, and 3 are visible in time but cite a relation outside the query’s own scope. Table II reports the first two apart and excludes the third, whose labels no scope-respecting operator can satisfy; the undifferentiated macro-average (0.722 for LGR) obscures all three.

TABLE II

FROZEN-PILOT EVIDENCE QUALITY BY LABEL REGIME (SECTION IV-A). “SATISFIABLE” TASKS ARE THOSE WHOSE GOLD EVIDENCE SET THE ADMISSION CONTRACT PERMITS AN OPERATOR TO RETURN; THREE FURTHER TASKS ARE SATISFIABLE IN TIME BUT OUT OF DECLARED SCOPE AND ARE EXCLUDED FROM BOTH COLUMNS. STALE AND LOSER ARE NON-EXPOSURE RATES, REPORTED BESIDE RECALL BECAUSE RETURNING NOTHING ALSO SCORES 1.000. ALL METHODS SHARE THE COMPLETED GATE, SO ACTION AGREEMENT IS 1.000 FOR EVERY METHOD.

Generator	Satisfiable (n=75)				Retro. (n=22)	
	F1	Rec.	Stale	Loser	F1	Rec.
Lexical top-k	0.392	1.000	0.460	0.747	0.335	1.000
Recency weighted	0.687	0.993	0.687	0.753	0.477	0.773
TTL-only	0.692	1.000	0.713	0.773	0.477	0.773
Temporal KG	0.723	1.000	0.680	0.907	0.509	0.636
Provenance KG	0.741	1.000	0.680	0.907	0.509	0.636
Ontology-versioned	0.676	1.000	0.393	0.907	0.509	0.636
GraphRAG-style	0.687	0.993	0.687	0.753	0.477	0.773
Event-sourced	0.482	0.540	0.727	0.827	0.318	0.318
LGR	0.963	1.000	1.000	1.000	0.000	0.000

On the 75 satisfiable tasks LGR attains eligible-assertion F1 of 0.963 (precision 0.949, recall 1.000) against 0.741 for provenance-aware KG retrieval, the strongest baseline: a paired bootstrap difference of +0.222, 95% interval [+0.193, +0.251] over 1,000 resamples. Its stale and contradiction-loser non-exposure are 1.000, exceeding the strongest baseline per metric by +0.274 [+0.220, +0.333] and +0.093 [+0.047, +0.147]. Recall accompanies each non-exposure rate, since returning nothing also scores 1.000; perfect recall on this stratum shows LGR withholds stale evidence, not all evidence.

On the 22 retrospective tasks LGR scores 0.000 on F1 and recall, while baselines reach 0.32–0.51 by exposing records not yet visible. This favors neither: those labels ask for evidence the contract forbids at that instant, so the stratum measures label semantics, not retrieval quality, and no credit is claimed on it. The 3 out-of-scope tasks are a label defect rather than a regime—their gold sets cite relations the query never requested. Correcting either group is benchmark work this release does not do; the partition is reported so that neither is silently averaged into a headline.

The partition is derived from the same admission contract the operator implements, so it is not an independent test of that contract, and the comparison on it is post-hoc on a split chosen after seeing the labels. Comparisons use whichever baseline is strongest per metric, with none excluded, and intervals stay descriptive (Section VII).

Every generator reaches exact action-set agreement of 1.000 on the 40 action tasks, because the common gate, not its top-k context, evaluates the action. One outlier is informative: ontology-versioned retrieval has the worst satisfiable-stratum stale non-exposure (0.393, below lexical top-k at 0.460), because its version-compatibility mapping widens matching to old-ontology names and surfaces disproportionately superseded assertions with no lineage check—version-compatible retrieval is counterproductive on staleness unless composed with temporal and lineage checks. Token and latency figures are archived with the artifact and support no efficiency conclusion.

B. Pre-specified multiworld evaluation

The validation protocol selected provenance-aware KG retrieval as the strongest local comparator by mean eligible-evidence F1 across the two validation worlds. On the ten untouched test worlds, comprising 743 primary tasks, LGR averages 0.961 F1 (world-level sample standard deviation 0.007), while the selected comparator averages 0.746 (standard deviation 0.009). The paired mean difference is +0.215, with a 95% world-bootstrap interval of [+0.212, +0.218]; every individual world difference is positive (Fig. 2), and LGR has recall and both non-exposure rates at 1.000 in every test world. These results establish consistency across seeds within one generator family, not transfer to independently authored scenarios or superiority over the published systems of Section II.

A custom governance composition, written separately and importing neither the LGR operator nor the generator, applies the same temporal, version, provenance, conflict, policy, workflow, and target-binding tables. Its admission path binds guard witnesses to one target and blocks actions that depend on an unresolved critical relation; separate two-account and unresolved-conflict regression cases exercise both safeguards. It reproduces LGR’s resolved evidence on all 1,500 tasks across development, validation, and test worlds and reproduces the table-derived action sets on all 635 action tasks. Equality is the informative result: the tested behavior can be assembled from conventional checks when their information, ordering, and fail-closed semantics are made identical. The empirical claim is therefore not that LGR defeats every reasonable composition; its contribution is the explicit operator contract, trace, and conformance surface.

C. Reference-implementation scaling and ablations

A scaling harness measures the unindexed Python reference implementation under two stress modes that separate total graph size from query-neighborhood size: in *unrelated growth* the completed dependency closure stays at 11 assertions while the snapshot grows to 100,000; in *dense growth* nearly every added assertion enters the closure. End-to-end p50 at 100,000 assertions is 0.55 s under the first and 2.98 s under the second, so latency tracks closure size rather than snapshot size—a property of this implementation’s full-snapshot scans and in-memory conflict grouping, not evidence of production-scale throughput. Full timings, percentiles, and per-stage digest costs are released with the artifact.

Figure 3 plots every nonzero ablation effect, separating individual controls from two explicit interactions. Trust ranking and conflict resolution each lower eligible F1, loser non-exposure, and action-set agreement; workflow-instance scope lowers F1 alone. Two results are interactions and are not attributed to either control alone. Disabling record-time visibility alone lowers stale non-exposure, while disabling valid time alone changes no aggregate metric, because record visibility still excludes the retrospective evidence; only the combined bitemporal ablation raises macro F1, and it does so by trading satisfiable-stratum F1 (0.963 to 0.907) and stale non-exposure

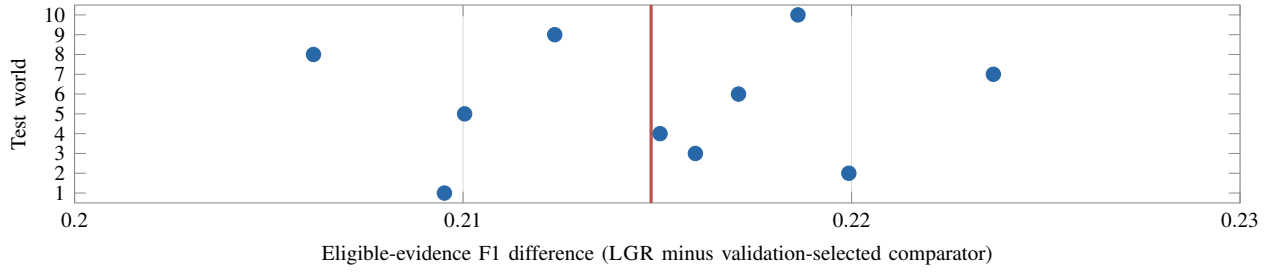


Fig. 2. Paired F1 differences for all ten pre-specified test worlds. The vertical line marks the equal-weight mean (+0.215); no test world was excluded. Worlds differ by generated histories and task draws but share one generator family.

(1.000 to 0.364) for retrospective F1. Likewise, action-target binding has no aggregate effect on this split by itself—though the constructed two-account case in Section VI shows it decides an unsafe admission—and the 15 action mismatches appear only when it and workflow-instance scope are disabled together.

D. Zero-effect ablations, masking, and decisive strata

The lifecycle, ontology-version, policy-version, and super-session ablations change no aggregate metric, yet these are features the benchmark exists to stress. A released masking audit shows the null is structural rather than sampling luck: each of these strata is fully covered by another filter, so toggling any of the four flags changes eligibility membership nowhere, as an executable check confirms. The release therefore adds ten decisive counterfactual tasks that one predicate alone decides—retraction without replacement (4), a replacement from a lower-authority source (3), and a wrong-regime policy fact with unclosed validity (3)—on each of which the full operator excludes the target for the intended sole reason while the ablation exposes it, twice admitting an effectful refund the operator blocks. The zero deltas are a property of overlapping strata: redundancy the decisive strata show to be load-bearing.

Seven further decisive cases separate controls the original broad flags coupled: record visibility from valid time, provenance-activity resolution from source eligibility, workflow-instance scope from action-target binding, and source eligibility from relation-specific trust ranking. In each eligibility case the full configuration emits exactly the named reason and disabling only its flag removes it. The trust case changes the winner from an older higher-authority record to a newer lower-authority one; the target-binding case turns a blocked refund into an invalid admission supported by another account. These constructed cases demonstrate mechanism sensitivity, not prevalence or an aggregate effect size.

VI. EXECUTABLE CONFORMANCE EVALUATION

The implementation loads the versioned governance tables of Section III, completes declared dependencies, evaluates every action through one default-deny function, resolves query-normalized conflicts by relation-specific trust and descending record time (ties unresolved, no assertion-ID surrogate), and records the completion manifest, witnesses with their selected rule and transition, blocking reasons, digest, and checkpoint.

All 57 checks pass. Thirty-four operator cases span 29 declared requirement areas—the eight eligibility predicates, normalization and split handling, conflict resolution and ties, registry and guard gating, trust-table validation, action target and scope, query mode, and decision-record replay and digest—while 5 generator fixtures cover authority before recency, recency within a tier, incompatible top ties, and equivalent top assertions; 13 label-validator mutations cover omitted, absent, malformed, and corrupted verdicts, witnesses, and reasons; and 5 comparator cases check evidence and action equality. The full test-to-requirement mapping ships with the artifact.

Thirteen cases are constructed counterexamples the frozen split cannot exercise at all, because every action task carries one coherent case in scope, no audit task carries candidate actions, and no released mapping is ambiguous under a task’s scope. Each pins an admission property on which the pilot’s aggregates are silent. The most consequential is cross-principal witnessing: with two accounts in scope—one suspended with an approved case, one active—nothing in the aggregate distinguishes a gate that refuses the first account’s refund from one that authorizes it with the second account’s subscription status. A related pair pins the *basis* for that binding: a case’s participants are read from the membership its own workflow records declare, so an unrelated account is excluded whether or not its identifier resembles the case’s, and a case retains its members when its identifier is renamed. The rest cover an unresolvable action target, a guard dependency discarded by a narrowed retrieval scope, an effectful action admitted under a historical query, split and renamed mappings, unresolvable and unregistered provenance, and malformed timestamps. Every reported number is identical whether these properties hold or fail: a benchmark can be entirely silent about a soundness property that a single counterexample settles.

Label evidence is separated from generation: the validation-only checkers import neither operator nor generator, write nothing, and fail on disagreement. The evidence checker independently derives contract-permitted candidates from the assertion snapshot, rejects malformed temporal fields, verifies every duplicated stage-label view, and detects a missing required answer. The action checker consumes that independently derived resolved set—not the supplied gold evidence—before re-deriving all 60 action verdicts, witnesses, and blocking reasons, including target binding and critical-conflict blocking.

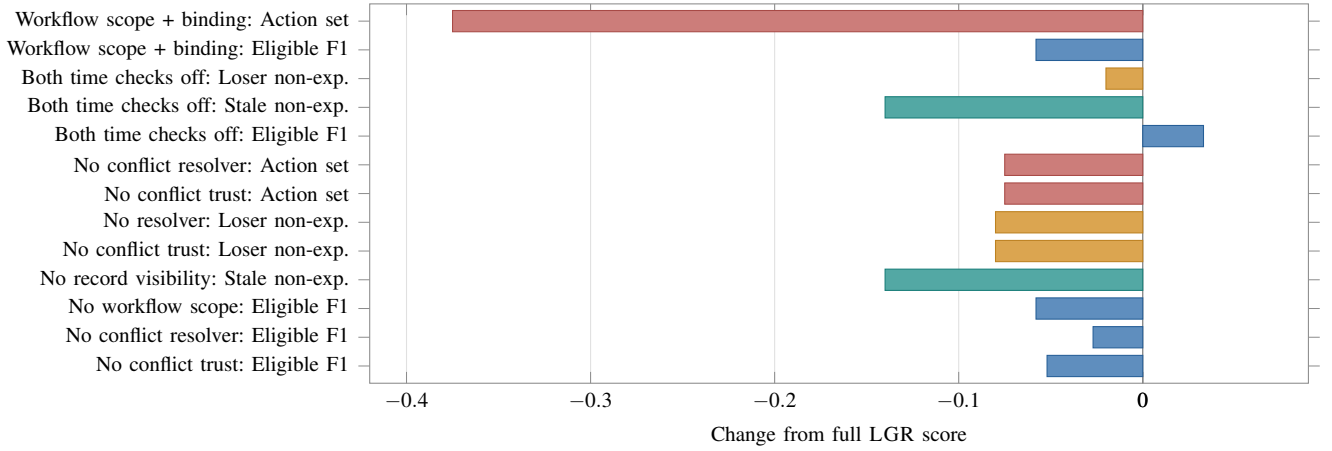


Fig. 3. Nonzero isolated-control changes and two explicitly combined interactions on the frozen pilot (ablated minus full LGR, whole split). “Both time checks” means record-time visibility and valid time; the top two rows jointly remove workflow-instance scope and action-target binding. Every nonzero feature-metric change is plotted; the omitted pairs changed nothing to three decimals (Section V-D).

The evidence partition remains 75 permitted, 22 retrospective, and 3 out-of-scope test tasks with no failure in the permitted group. These checks consume the same generated worlds and tables, so they establish consistency with those tables, not their organizational correctness.

Passing these cases is not implementation conformance: each is one branch on a constructed or benchmark input, establishing neither exhaustive verification nor the absence of a further counterexample. The replay check detects changed local inputs but tests no durable store, concurrency, or log recovery, and the label check verifies labels against the tables, not the tables against an organization. SHACL or XACML [28], [29] are possible implementation substrates, but no claim of running either engine is made.

VII. THREATS TO VALIDITY AND LIMITATIONS

Construct validity. Exact action-set agreement measures whether the evaluator and a separately implemented interpreter reach the same result from the same closed-world tables—not that the tables are complete, that an LLM will follow the packet, or that an executed action is safe. Eligible F1 measures agreement with generated labels and is reported only within a declared regime. Non-exposure stays weaker than correct reason-coded suppression even when paired with recall. The composed comparator’s exact agreement shows that the operator is implementable from its contract, but cannot validate a contract whose inputs both implementations share.

Internal validity. The generator, tables, canonical evidence, composed comparator, and both label interpreters share scenario assumptions. Separate implementations reduce common-code errors but not common-specification errors. The masking audit quantifies how overlapping strata limit causal attribution, and the decisive strata and admission counterexamples restore it only in constructed worlds; untested properties may remain. The version-3 test seeds were fixed before their results were inspected, and every world was retained, but all were produced by one generator family.

Conclusion validity. The original task-level bootstrap is post-hoc within one generated world and does not make its tasks independent. The extension selects one comparator on two validation worlds and resamples ten whole test worlds, so its interval measures seed variation within the generator more appropriately, but ten worlds remain a small engineering sample rather than a power-based design. No multiplicity adjustment is applied to secondary metrics, and the results do not cover alternative generator families, independently authored tables, or materially different trust orders.

External validity. EvoMem-Enterprise is controlled and synthetic, representing neither naturally evolving corpora, extraction errors, organizational exceptions, adversarial data, nor local legal requirements; multiple seeds improve robustness to generated variation, not to these omitted conditions. A protocol and blank template for independently authored scenarios are included, but no external contributor or real LLM evaluation is reported.

Open work. The same-information composition and ten-world evaluation close two earlier gates but narrow the claim: conventional temporal, graph, and policy checks reproduce the tested operator when given the same tables and fail-closed order. Still open are independently authored scenarios and labels, alternative generator families and trust orders, a persistent indexed implementation under concurrent updates, durable recovery tests, and a fixed-version real-LLM evaluation reporting accuracy, invalid-admission rate, citation support, latency, tokens, and cost separately.

Limitations. The formal results are conditional on a complete declared dependency closure; complete and correct registry, policy, workflow, ontology, provenance, trust, and relation-semantics tables; and a faithful implementation. They guarantee internal properties of the specified operator, not factual truth, legal compliance, fairness, privacy, or authorization beyond the modeled controls. The scaling harness neither predicts a graph database nor exercises concurrent writes. The near-

term use is decision support with explicit provenance and human confirmation for effectful operations; deployment must independently validate table ownership, update procedures, access controls, snapshot immutability, audit retention, recovery, and the packet-LLM interaction, none of which the local digest and replay tests discharge.

VIII. CONCLUSION

LGR makes action admissibility a first-class KG systems problem: evidence must become eligible before it becomes persuasive, and an effectful action must become registered, targeted, and governed before it becomes available. On the 75 frozen-pilot tasks whose gold evidence the contract permits an operator to return, it reaches 0.963 eligible-evidence F1 against 0.741 for the strongest implemented local baseline at 1.000 recall with perfect stale and conflict-loser non-exposure. Across ten pre-specified test worlds and 743 primary tasks, the corresponding means are 0.961 and 0.746, a paired difference of +0.215 with a world-bootstrap interval of $[+0.212, +0.218]$. The claim is consistency under generated seed variation, not external validity.

The strongest lesson is that aggregate scores and conformance answer different questions. Admission properties such as cross-target witnesses, unresolved targets, historical query mode, split mappings, and provenance resolution can remain invisible to a benchmark average, while targeted counterexamples make them observable. Conversely, the same-information custom composition matches all tested LGR evidence and action outputs, so no empirical advantage should be claimed over a faithfully composed implementation. The defensible contribution is a falsifiable contract, an executable trace, and evidence that its behavior is reproducible across the tested generated worlds; independently authored scenarios, durable indexed replay, and real-model evaluation remain necessary.

DATA AND CODE AVAILABILITY

All results reported here were produced from the tagged release v3.0.0, archived at [doi:10.5281/zenodo.22651556](https://doi.org/10.5281/zenodo.22651556) and mirrored at <https://github.com/amith-reddy/lgr-action-safe-enterprise-kg>. Cite the DOI, not the default branch, which will advance. The release contains the frozen benchmark, the versioned labeling specification, all 15 multiworld datasets and per-task outputs, validation-only checkers, conformance and mutation tests, the standalone composed comparator, raw scaling timings, fixed protocols, the full test-to-requirement mapping, and a no-mutation reproduction command. Code is MIT-licensed and the generated benchmark data is released on the same terms.

ACKNOWLEDGMENT

The author used generative-AI tools for outline refinement and draft preparation, independently verified the final manuscript's claims, equations, results, and references, and assumes responsibility for its content.

REFERENCES

- [1] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.
- [2] D. Edge *et al.*, "From local to global: A graph RAG approach to query-focused summarization," arXiv:2404.16130, 2024.
- [3] X. Zhu *et al.*, "Knowledge graph-guided retrieval augmented generation," in *Proc. NAACL*, 2025.
- [4] J. Yu *et al.*, "Can knowledge-graph-based RAG really retrieve what you need?" in *Proc. NeurIPS*, 2025.
- [5] X. He *et al.*, "G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering," in *Proc. NeurIPS*, 2024.
- [6] *Moffatt v. Air Canada*, 2024 BCCRT 149, Feb. 2024.
- [7] B. Jiménez Gutiérrez *et al.*, "HippoRAG: Neurobiologically inspired long-term memory for large language models," in *Proc. NeurIPS*, 2024.
- [8] P. Rasmussen *et al.*, "Zep: A temporal knowledge graph architecture for agent memory," arXiv:2501.13956, 2025.
- [9] Z. Y. Zhang *et al.*, "Respecting temporal-causal consistency: Entity-event knowledge graph for retrieval-augmented generation," in *Proc. EACL*, pp. 2017–2054, 2026.
- [10] J. Han *et al.*, "RAG meets temporal graphs: Time-sensitive modeling and retrieval for evolving knowledge," arXiv:2510.13590, 2025.
- [11] R. Yang *et al.*, "Beyond single pass, looping through time: KG-IRAG with iterative knowledge retrieval," arXiv:2503.14234, 2025.
- [12] R. Trivedi *et al.*, "Know-Evolve: Deep temporal reasoning for dynamic knowledge graphs," in *Proc. ICML*, 2017.
- [13] D. Li *et al.*, "T-GRAG: A dynamic GraphRAG framework for resolving temporal conflicts and redundancy in knowledge retrieval," in *Proc. ACM Int. Conf. Multimedia (MM)*, 2025.
- [14] S. Liu, Y.-M. Shang, and X. Zhang, "TruthfulRAG: Resolving factual-level conflicts in retrieval-augmented generation with knowledge graphs," *Proc. AAAI Conf. Artif. Intell.*, vol. 40, no. 38, pp. 32168–32176, 2026.
- [15] S. Wang *et al.*, "LedgerRAG: Governance-driven agentic chain of retrieval for dynamic knowledge scenarios," *Electronics*, vol. 15, no. 7, Art. 1376, 2026.
- [16] X. Su *et al.*, "GRAG4PM: Graph retrieval augmented generation framework adapted for process mining," *Applied Sciences*, vol. 16, no. 10, Art. 5152, 2026.
- [17] T. Martin and S. Gilchrist, "A2JIRAG: Process-aware retrieval-augmented generation for public legal information systems," in *AIDA2J Workshop, ICAIL*, 2026.
- [18] C. S. Jensen and R. T. Snodgrass, "Temporal data management," *IEEE TKDE*, vol. 11, no. 1, pp. 36–44, 1999.
- [19] K. Kulkarni and J.-E. Michels, "Temporal features in SQL:2011," *SIGMOD Rec.*, vol. 41, no. 3, pp. 34–43, 2012.
- [20] C. Gutierrez *et al.*, "Introducing time into RDF," *IEEE TKDE*, vol. 19, no. 2, pp. 207–218, 2007.
- [21] D. Huwiler, K. Stockinger, and J. Fürst, "VersionRAG: Version-aware retrieval-augmented generation for evolving documents," arXiv:2510.08109, 2025.
- [22] X. Dong *et al.*, "Knowledge Vault: A web-scale approach to probabilistic knowledge fusion," in *Proc. ACM SIGKDD*, 2014.
- [23] G. Antoniou *et al.*, "Representation results for defeasible logic," *ACM TOCL*, vol. 2, no. 2, pp. 255–287, 2001.
- [24] Z. Xiang *et al.*, "GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning," arXiv:2406.09187, 2024.
- [25] Object Management Group, *Business Process Model and Notation (BPMN)*, Version 2.0.2, Dec. 2013.
- [26] T. Lebo *et al.*, Eds., *PROV-O: The PROV Ontology*, W3C Recommendation, Apr. 2013.
- [27] O. Hartig *et al.*, Eds., *RDF-star and SPARQL-star*, W3C Community Group Final Report, Dec. 2021.
- [28] H. Knublauch and D. Kontokostas, Eds., *Shapes Constraint Language (SHACL)*, W3C Recommendation, Jul. 2017.
- [29] OASIS, *eXtensible Access Control Markup Language (XACML)*, Version 3.0, Jan. 2013.
- [30] M. Fowler, "Event sourcing," martinowler.com, 2005.
- [31] S. Huang *et al.*, "Temporal graph benchmark for machine learning on temporal graphs," in *Proc. NeurIPS D&B*, 2023.